A Validated Learned Market Model for Sample-Efficient Battery Storage Arbitrage

Nicholas Lai Jin Yung
220947084
David Mguni
MSc Artificial Intelligence
School of Electronic Engineering and Computer Science
Queen Mary University of London

***Abstract*—Training a reinforcement-learning controller for battery-storage arbitrage model-free demands years of once-a-day market data, and any policy tuned on historical prices must eventually face an unseen regime. We exploit a structural property of the problem. A battery small relative to the grid is a *price-taker*, so its environment decomposes into battery dynamics known exactly and an exogenous market price process observed for free, once per day, regardless of policy behaviour. We therefore learn only the market, as an ensemble of hour-autoregressive networks admitted into training only after passing a statistical validation gate against held-out data, and train a Soft Actor-Critic agent Dyna-style inside it, its replay buffer seeded by planner demonstrations so the physical asset never executes an untrained action. Under a fixed real-data budget, this gated model nearly matches an unlimited-data model-free ceiling and clearly beats a replay-only control given identical data and gradient budget. It's the *validated* model, not additional compute, that a single season of real data buys. We then map three ways this decomposition stops paying. The policy degrades gracefully, a floor rather than a solution, under unseen transient shocks. A nightly refit-and-replan adaptation loop fails to beat a frozen pre-shift policy under persistent regime shifts, because incremental refitting drives its market model outside the validation gate within three nights, on stationary data and weeks before the shift arrives. And a physics-informed battery surrogate is simply not worth it on this problem, losing to a plain network at every realistic data budget, with an ablation against deliberately wrong physics showing its residual carries no information at those budgets. All results are normalized against exact optimization oracles in a calibrated, deterministic merit-order market simulator enabling exact paired counterfactual evaluation.**

***Index Terms*—reinforcement learning, Dyna, model-based reinforcement learning, battery energy storage, electricity price arbitrage, non-stationarity, physics-informed neural networks**

I. Introduction

A. Background and Motivation

The global transition away from fossil fuels and towards sustainable energy sources is reshaping how modern power grids are planned and operated. Renewable technologies, particularly solar and wind, are essential for decarbonisation, but they also introduce a major operational challenge because their output is intermittent (International Energy Agency, 2023). In practice, renewable generation doesn't always align with demand. Solar and wind output can be high when demand is relatively low, while the grid may experience stress during evening peaks or periods of low wind availability.

Grid-scale Battery Energy Storage Systems (BESS) provide one way to address this mismatch between generation and demand. By charging during periods of surplus generation and discharging when demand is high, batteries act as an essential energy buffer helping stabilise the grid against volatility and supporting higher levels of renewable penetration (Aneke and Wang, 2016). This makes BESS an increasingly important part of future electricity systems.

However, operating BESS effectively isn't straightforward. The economic viability of a battery often depends on electricity price arbitrage, where the system charges when prices are low and discharges when prices are high. A policy that focuses only on short-term profit may cycle the battery aggressively, which can accelerate capacity fade and reduce its State of Health (SoH) (Xu et al., 2018). The long-term value of energy storage depends on control strategies that account for ageing mechanisms alongside revenue.

Compounding this challenge, the battery operates in a shifting market where prices are influenced by a variety of factors creating regimes that may not have been encountered during the training of a control policy. Europe's 2022 gas-price crisis is a concrete instance, moving both the level and the shape of electricity prices well outside any recent training window (International Energy Agency, 2022). This raises a question that goes beyond whether reinforcement learning can learn a good arbitrage policy, can the learned controller remain effective when the market shifts to a regime it has never seen?

B. The Problem

Reinforcement learning (RL) is a suitable framework for battery arbitrage because the problem is sequential by nature. At each timestep, the controller must decide whether to charge, discharge, or remain idle, while accounting for uncertain electricity prices and the long-term impact of degradation. This fits naturally within the Markov Decision Process (MDP) formalism. Recent work has shown that deep RL agents can learn profitable energy-storage arbitrage strategies and can outperform hand-crafted heuristics (Cao et al., 2020), while Soft Actor-Critic (SAC) is a common choice for continuous-control problems because of its off-policy learning and entropy regularisation (Haarnoja et al., 2018).

However, applying RL to battery arbitrage is complicated by an environment that shifts under the agent. This domain presents three core challenges.

Sample inefficiency. Model-free algorithms such as SAC learn mainly through environment interaction. In this project, one training episode is 14 days of hourly dispatch decisions (336 steps), and training SAC from scratch to convergence takes hundreds of thousands of such steps (Section VI-D) on the order of years of once-a-day real market data if learned model-free and from scratch.

Constraint violations during exploration. Exploration, particularly early in training, can produce physically infeasible actions. Examples include discharge requests that would push the State of Charge (SoC) below its lower operating limit or charge requests that would exceed the upper limit.

Market regime shifts. Battery arbitrage depends on a market process that is itself non-stationary. Electricity prices reflect seasonal demand patterns, renewable availability, weather conditions, and fuel costs, all of which can drift gradually or shift abruptly. A policy calibrated to one regime can encounter very different price spreads and cycling incentives after such a shift.

Together, these challenges motivate a control approach that can learn from few real market days, never explores untrained on the physical system, and adapts to persistent regime shifts at a speed set by days of observation rather than weeks of on-asset fine-tuning.

C. Approach

The objective is fixed from the beginning as maximizing arbitrage profit and minimizing battery degradation, with wear priced at the cost of the health it consumes. To investigate how to pursue that objective from little real data, the dissertation builds a calibrated, fully deterministic market-and-battery simulator as a controlled testbed and studies a family of methods on it. The central method is Dyna where a SAC agent that trains inside a learned model of the environment's dynamics rather than on real interaction alone. Those dynamics split naturally into a market half and a battery half, the dissertation applies Dyna with both halves as the observation, with questions that follow from doing so. Learning the market half turns a season of freely-observed price days into unlimited imagined training days, and the recurring theme throughout the dissertation is the extent to which this approach achieves the performance of model-free arbitrage while relying on substantially less real data. The same structure is then probed from other angles; whether training off-asset keeps exploration safe on the physical battery, whether the framework degrades gracefully under transient shocks, whether a nightly refit-and-replan loop lets it track persistent regime shifts, and whether the battery is better learned than assumed.

This decomposition is motivated by a simple structural observation. A battery operating as a price-taker, the standard modelling assumption for storage small relative to the system it trades in (Sioshansi et al., 2009), interacts with two distinct domains; a deterministic physical system and a stochastic market environment.

We therefore decompose the world model accordingly.

- **The asset physics are known and deterministic, by default.** The State of Charge (SoC) and State of Health (SoH) evolve through well understood electrochemistry, including round-trip efficiency, throughput-based cycling degradation, and calendar ageing.
- **Only the market is stochastic and learned, in the default setup.** Rather than trying to model complex grid-level market drivers, the agent only requires a generative model of 24-hour price vectors. We implement this as an ensemble of hour-autoregressive recurrent networks conditioned on calendar features and historical prices.

This decomposition means only the dynamics of the market half ever needs to adapt to the changing prices, while the battery half remains fixed. A model-free agent gathers at most 24 real transitions per day, no matter how urgently its policy needs updating. The generative model turns those same 24 observed prices into unlimited imagined training days overnight, so the effective sample rate is bounded by compute rather than by the calendar.

D. Research Questions and Contributions

Every investigation in this dissertation shares the premise that a price-taking storage asset's environment decomposes into a battery and an exogenous market price process. A Dyna-style SAC agent trains using imagined transitions validated against held-out data (the market case) or admissibility by construction (the battery case). Because the learned market process is exogenous, the policy's actions can't steer it into self-confirming error, so imagination can run over full 24-step days rather than short branched rollouts. Each investigation below is framed as a question and reported empirically with its finding, positive or negative.

1) **Under a fixed data budget, is it a validated world model, not more compute, that buys sample efficiency?** The market model, an ensemble of hour-autoregressive networks, is admitted to policy training only if a pre-training gate certifies its imagined days as within the calibration range of the market. At a fixed real-data budget, Dyna training inside this gated model nearly matches an unlimited-data model-free ceiling and clearly beats a replay-only control trained on the identical budget with unlimited extra gradient epochs, which rules out "more updates" as the explanation (Section VII-A).
2) **Does training in imagination with planner demonstrations produce a deployed policy that requests fewer infeasible actions than one trained model-free?** No, evaluated on the real asset, the trained Dyna policy requests an SoC-bound-throttled action on 92.8% of steps, statistically indistinguishable from a policy trained entirely model-free (95.1%) or from replay alone (95.2%). The off-asset design doesn't carry over into a deployed policy that violates SoC bounds less often (Section VII-B).

3) **Does the validated world model degrade gracefully under transient shocks it never trained on?** The frozen dyna policy, an unlimited-data model-free policy, and a replay-only control, all trained purely on calm historical data with no shock exposure, are evaluated zero-shot against five unseen shocks on the same shocked days. Dyna clearly outperforms the replay-only control and closely tracks the unlimited-data policy shock for shock, on a real-data budget two orders of magnitude smaller, even edging it under the fuel shock. More calm-regime data doesn't buy shock robustness (Section VII-C).
4) **Can an in-imagination adaptation loop recover from persistent regime shifts?** A nightly refit-and-replan procedure is tested on a deployment where the market shifts partway through. It fails to beat simply freezing the pre-shift policy. The nightly refit's market predictions drift outside the same calibration range used to approve the original model, and never return to it for the rest of the deployment (Section VII-D).
5) **Does a physics-informed battery surrogate need fewer real transitions than a plain MLP to predict the battery's own dynamics?** Holding the market exact, the battery alone is learned from a transition log. On this smooth, low-dimensional map, the physics-informed network shows no robust sample-efficiency crossover over a plain MLP, not even in the small-data regime a PINN is meant to own (Raissi et al., 2019). The finding is negative, and an ablation that refits the network with deliberately wrong physics explains it: the residual carries genuine physical information only at the largest budgets, where breaking it costs about $4.9\times$ the error, and is inert below $N{=}50$, precisely the scarce-data regime that motivates a physics prior in the first place. The zero rate of physically impossible predictions usually claimed for such models is definitional here, imposed by an output clamp that any network can carry, so it isn't evidence for the physics prior either. On this problem the physics-informed approach isn't practical (Section VII-E).

II. Related Work

A. Reinforcement Learning for Battery Arbitrage

The application of RL to electricity market bidding and battery dispatch has attracted growing attention. Early work by Cao et al. (2020) demonstrated that deep Q-networks could learn profitable day-ahead bidding strategies, outperforming linear-programming baselines under price uncertainty. Subsequent studies have applied policy-gradient methods, such as Proximal Policy Optimization (PPO), and off-policy actor-critic architectures, such as Soft Actor-Critic (SAC), for real-time arbitrage. Among these, SAC has emerged as a strong default choice for continuous-action energy control problems due to its entropy regularisation and off-policy sample reuse (Haarnoja et al., 2018).

A central focus within this literature is the tension between short-term profit maximisation and long-term asset preservation. While model-free controllers can be trained on reward formulations that charge for physical battery degradation, they remain vulnerable to non-stationary environments. In modern energy markets, sudden shifts in supply and demand can strand a policy tuned to the old regime. Under standard model-free paradigms, adapting to a new regime requires extensive online retraining which is rate-limited by the market itself, yielding only 24 real hourly transitions per day, and during which the agent may execute suboptimal or physically damaging cycles on the asset. This dissertation studies robustness and adaptation to such shifts as its central question, rather than as an afterthought to a stationary benchmark.

B. Model-Based Reinforcement Learning and World Models

Model-based RL (MBRL) improves sample efficiency by learning a dynamics model $\hat{f}(s,a) \to s'$ and using it to generate additional training data. The Dyna architecture (Sutton, 1991) is the influential framework where the agent updates a dynamics model from real experience and generates synthetic transitions from it to augment the replay buffer. This approach has been shown to reduce the required number of real interactions by an order of magnitude in continuous-control benchmarks (Janner et al., 2019; Chua et al., 2018).

In the general setting, the quality of synthetic data is limited by model errors that *compound through the agent's actions* over multi-step rollouts motivating short branched rollouts from probabilistic ensembles (Janner et al., 2019) and uncertainty-aware planning (Chua et al., 2018). A key point of this dissertation is that the price-taker decomposition removes this failure mode and the learned process (daily prices) is exogenous, so the policy's actions can't steer the model into self-confirming error.

World-model approaches such as Dreamer (Hafner et al., 2020) learn latent dynamics and plan entirely in imagination. While powerful, these methods introduce substantial architectural complexity and are harder to interpret in safety-critical energy applications. The approach taken here retains a directly inspectable decomposition where exact battery physics plus a small generative model of prices whose fidelity is measured against held-out data before it's ever trusted.

C. Non-Stationarity and Adaptation in Reinforcement Learning

Reinforcement learning under non-stationary dynamics has been studied as worst-case planning against continuous drift (Lecarpentier and Rachelson, 2019), as a continual-learning problem balancing plasticity against forgetting (Khetarpal et al., 2022), and surveyed more broadly by Padakandla (2021); the supervised-learning literature on concept drift (Gama et al., 2014) contributes the detect-then-adapt pattern this work follows. The setting here is distinctive in two respects. First, the non-stationary component is *exogenous and fully observed* such that every day the market publishes the very quantity the model must track, so adaptation data accrues at a fixed rate for free, independent of the policy. Second, the asset constraint forbids on-policy re-exploration, so adaptation must run off-asset in order of refiting the small market model, retraining in imagination, deploying tomorrow. The dissertation's adaptation experiment measures how much this structure is worth relative to model-free online fine-tuning bound to the real stream.

D. Physics-Informed Surrogates for Dynamics Models

A separate line of work embeds known governing equations into the network itself. Physics-informed neural networks (Raissi et al., 2019) add a loss term penalising violation of the governing law at collocation points, which carry no labels, so the physics is meant to substitute for data and the model to reach a given accuracy from fewer measured samples. That promise has been carried into lithium-ion modelling, where physics-informed networks are reported to give stable degradation prediction and prognosis from limited cell data (Wang et al., 2024), into continuous control (Antonelo et al., 2024), and into the exact architecture used here, since Liu and Wang (2021) train a Dyna-style agent inside a physics-informed dynamics model. Together these motivate inverting this dissertation's default assumption, learning the battery half of the decomposition rather than assuming it, from a transition log short enough that an operator might plausibly have one. The scope of the original claim matters for reading the result. The residual is established where the target map is stiff, high-dimensional, or observed too sparsely for an unconstrained network to interpolate; a price-taking battery's step map is smooth, three-dimensional and cheap to label, so Section VII-E measures how far the sample-efficiency claim extends rather than whether it ever holds.

III. Synthetic Merit-Order Market Simulator

A. Justification for Synthetic Data

A synthetic, fully observable simulator is used for four reasons. It provides controlled experimentation, seeded reproducibility, unlimited paired evaluation episodes, and regime-shift scenarios that consume no randomness, so a shifted run and its calm same-seed counterfactual are bit-identical outside the shock window. Real historical data can't provide on-demand, repeatable regime shifts with a known counterfactual. The synthetic setting also makes the evaluation stronger. The hindsight optimum and the rolling-horizon oracle are exactly computable, so every policy can be scored as a fraction of the value actually attainable on the same days. The simulator is a controlled testbed, so the claims this dissertation defends are orderings and mechanisms established within it, not absolute profit levels, and no claims transfer to real markets without validation against measured data (Section VIII-B).

B. Demand

Prices form causally from an exogenous driver layer. Demand follows a heating-dominated European profile, driven by a calendar (day-of-year, weekends, holidays) and an Ornstein Uhlenbeck (OU) temperature process. Its seasonal base load is U-shaped over the year, with the heating peak midwinter and only modest summer cooling load, overlaid with daily shape, weekend, and holiday effects. This consumption pattern, pushed through a sloped merit order, is what produces the winter price premium. The premium *emerges* from demand meeting the supply curve rather than being painted onto prices.

C. The Generator Fleet

The fleet comprises solar (500 MW, the duck-curve driver), wind, must-run geothermal, a reservoir hydro plant that bids its opportunity cost as a function of reservoir fill (self-stabilising), and a deliberately granular thermal fleet made up of two coal units with minimum-run blocks and nine gas units spanning efficient combined-cycle plants through old open-cycle peakers. Solar and wind availability follow OU weather processes (irradiance and wind proxies), and the thermal units' fuel costs follow an OU fuel market (gas and coal prices). Shocks perturb the drivers, never prices directly, so every price movement has an economic mechanism behind it. The granularity is the point. Units of varying efficiency make the merit order slope, so seasonal demand moves the marginal cost. Negative prices emerge from subsidy-style negative renewable bids, coal min-run blocks, and must-run geothermal, mostly on sunny summer middays. Scarcity hours clear at the price cap and concentrate in winter and early spring, where high demand coincides with outages and low renewables.

D. Market Clearing

Each hour clears as a uniform-price merit-order auction. Every generator submits (price, quantity) offers, the clearing intersects the resulting supply curve with demand, and all dispatched capacity is paid the marginal price. One simulated day yields 24 prices plus dispatch by technology.

E. Calibration

The simulator is held to a statistical calibration contract, enforced by the test suite over a seeded simulated year. The median price is $\approx$ \$64/MWh, the winter median more than \$6 above the summer median and the winter mean more than \$15 above it, the evening peak above overnight prices, scarcity below 1.5% of hours, and negative prices rare but present. These thresholds were fixed by tuning the fleet and driver parameters until the simulated year showed the structure above, not by fitting to measured prices, so no value here is an empirical estimate. They serve as regression bounds, and a failure means the market drifted when a fleet or weather parameter changed.

F. Regime-Shift Scenarios

Regime shifts are expressed through five scenario types namely *ColdSnap* (heating-demand surge), *HeatWave* (cooling demand plus thermal derating), *FuelShock* (multiplicative gas-price step), *Drought* (reservoir inflow collapse), and *PlantOutage* (a named unit removed from the fleet). Scenarios perturb *drivers* inside a day window, never prices, and consume no randomness, so the same seed with and without a scenario differs only through the shock's causal consequences. The same scenario serves both experimental roles. Short windows give the unseen *transient shocks* of the robustness study (Section VI-B), and windows extended to the end of the run give the *persistent regime shifts* of the adaptation study (Section VI-C).

IV. Problem Formulation: The Arbitrage MDP

A. Battery Physics

The battery (100 kWh, 50 kW, 90% round-trip efficiency, split evenly so each direction carries a $\sqrt{0.9} \approx 94.9\%$ one-way efficiency) tracks two states. SoC is expressed as a fraction of the *current degraded* capacity $C_{\text{eff}} = C_{\text{nom}} \cdot \text{SoH}$, and SoH declines through two mechanisms, cycling and calendar aging,

$$\Delta\text{SoH}_{\text{cyc}} = E_{\text{thru}} \cdot \kappa_{\text{cyc}} \cdot \left(1 + s \cdot \frac{|P|}{P_{\max}}\right), \tag{1}$$

$$\Delta\text{SoH}_{\text{cal}} = \kappa_{\text{cal}} \cdot \left(0.5 + \overline{\text{SoC}}\right) \cdot \Delta t, \tag{2}$$

where E_{thru} is the cell-side energy throughput of the step and κ_{cyc} is calibrated so that a full cycle life (5,000 equivalent full cycles) at low power takes SoH from 1.0 to the end-of-life floor of 0.8. The stress factor s makes high-power cycling cost more per kWh, standing in for the empirically established super-linear damage of aggressive cycling (Xu et al., 2018). Calendar ageing ticks every hour regardless of action and is calibrated to a 15-year rest life at mid SoC while a full battery ages three times as fast as an empty one (the $(0.5 + \overline{\text{SoC}})$ factor ranges from 0.5 to 1.5). SoH floors at 0.8 and feeds back into usable capacity; stored energy is conserved as capacity shrinks (SoC is re-expressed against the smaller capacity). Requested power is clipped to power limits and to what the SoC bounds ($[0.05, 0.95]$) allow, so every executed action is physically feasible by construction.

B. State Space

The observation is 32-dimensional and comprises 8 scalars (SoC, SoH, sine/cosine of hour-of-day, sine/cosine of day-of-year, weekend and holiday flags) followed by the *next 24 hourly day-ahead prices*, log-normalized. The market prices aren't hidden as day-ahead markets publish them in advance, and the rolling-horizon oracle (Section V-A) shows this window carries 99.9% of the attainable value. Weather and fuel states are deliberately *not* observed. They influence reward only through prices, which are already known exactly over the decision-relevant horizon.

C. Action Space

A single continuous action $a \in [-1, 1]$ maps to a power command $a \cdot P_{\max}$, where positive charges, negative discharges, and zero idles. Infeasible commands are clipped by the physics (Section IV-A) rather than penalised, so the policy can't damage the battery by violating operating limits; degradation, not infeasibility, is the cost channel.

D. Reward

The reward monetizes degradation at replacement value rather than weighting it by a free parameter,

$$r_t = \left(\pi_t - \Delta\text{SoH}_t \cdot c_{\text{repl}} \cdot C_{\text{nom}}\right) \cdot \sigma, \tag{3}$$

where π_t is the arbitrage cash flow of the hour (buying energy at the cleared price when charging, selling when discharging, with conversion losses applied on each side), ΔSoH_t is the health lost this step from both ageing pathways, $c_{\text{repl}} = \$250$/kWh is the replacement cost, and σ is a fixed reward scale. There's no tunable degradation weight and wear is priced at what it costs to buy back, so "minimizing degradation" is a single economically meaningful objective where every policy in the evaluation optimises the same dollars. Calendar ageing makes doing nothing strictly costly (an idle policy loses money), which anchors the low end of the benchmark ladder.

E. Episodes and Units

Episodes are 14 days (336 hourly steps) with randomised initial SoC and SoH and a burn-in of simulated days so weather and fuel states decorrelate from their initial values. The market operates in MW/MWh, the battery in kW/kWh; settlement converts between them.

V. Methodology

A. Oracles and Baselines

Every result is compared against four reference policies.

- **Hindsight optimum.** The best schedule possible, found by dynamic programming over a discretised grid of State-of-Charge values, using the actual prices for the whole episode. It's a valid upper bound because the battery is a price-taker, so its trading can't change the prices it plans against. Perfect-foresight dynamic programming under exactly this assumption is the established way the arbitrage value of storage is bounded (Sioshansi et al., 2009). This defines 100% of the value any policy could attain.
- **Rolling-horizon oracle.** The same dynamic program, but limited to what the agent actually knows at each step (the next 24 known prices), and re-run every hour. It reaches 99.9% of the hindsight optimum. This tells us the 24-hour window already contains almost all the information needed, so any gap left by a learned policy is a failure to learn, not a lack of information. A test pins its cost model to the real battery physics, so "optimal" really means optimal against the true dynamics. In the adaptation experiment it also serves as the *instant-adaptation baseline*, since it replans on each day's observed prices and so adapts immediately by design.
- **Heuristic.** A simple price-threshold rule that charges when the price is in a low percentile of the known window and discharges when it's in a high one. It stands in for a trusted human operator.
- **Idle.** This arm does nothing at all. It still loses money because the battery ages over time (calendar ageing), so it marks the floor.

Most of the arbitrage value comes from a few scarcity weeks, so profit varies a lot from one market week to the next. To keep comparisons fair, each one uses at least 20 paired episodes that share the same random seed, and results are reported as a percentage of the hindsight optimum rather than in raw dollars.

B. The Learned Market Model

The world model is a generative model of a single market day. It produces 24 hourly prices (log-normalized) from a 7-dimensional context made up of four calendar features (position in the year, weekend, holiday) and three summary statistics of the previous day's prices (log-normalized mean, minimum, and maximum). Generation is *hour-autoregressive*, which means the model produces the day one hour at a time. The context sets the initial hidden state of a gated recurrent unit (GRU, Cho et al., 2014), which then steps through the 24 hours; each hour's price is drawn from a Gaussian that depends on the hours already generated. To capture the model's own uncertainty, we train an ensemble of $K = 5$ networks, each fit on a bootstrap resample of the observed days, following the standard practice of treating disagreement across independently fitted networks as a predictive-uncertainty estimate (Lakshminarayanan et al., 2017; Osband et al., 2018). Each imagined episode picks one ensemble member and generates day after day, feeding each generated day back as the context for the next.

We also record two architectures that didn't work. A feedforward model that generates all 24 hours in one shot as independent Gaussians (and a low-rank variant that allows limited hour-to-hour correlation) fails the validation gate. Because these models treat the 24 hours as independent, they can't represent the fact that scarcity comes in blocks of several consecutive expensive hours. To reproduce those hours at all, they have to give every hour a fat tail, which makes the daily price spread far too wide. The hour-autoregressive model avoids this. Once it effectively "decides" that a day is a scarcity day, it stays on that path for the rest of the day.

C. The Validation Gate

Before it's used for any policy training, the fitted ensemble must pass a statistical gate. We generate synthetic years from the model and score them on the same calibration statistics the simulator itself is tested against, so the learned model has to meet the same standard as the real market. For checking price spread, the reference is a held-out real year generated from a seed that isn't part of the training data.

The criteria cover the market properties that arbitrage value depends on. The synthetic median price must fall within the calibrated band. The winter price premium must show up in both the median and the mean, and evening prices must be higher than overnight prices. The mean daily spread must be within 30% of the held-out real year, which catches both models that invent too much spread and models that smooth it away. Finally, scarcity hours must stay below 1.5% of all hours, and negative-price hours must occur but stay below 15% of all hours, so a model can't pass while getting wrong the exact hours where most of the value is.

A model that fails any criterion isn't used and the Dyna training script refuses to run unless it's given a model that has passed the gate. In standard model-based RL, trust is usually controlled by limiting how far the model is rolled out, because prediction errors compound as the policy acts. Here they can't, because the policy can't affect prices, so the only question is whether the generated days have the right distribution, and that can be checked *before* any RL compute is spent.

D. Dyna Training with Planner Demonstrations

The Dyna agent is a standard SAC agent trained inside the learned market, with one addition. Before training begins, we fill part of the replay buffer with transitions from the rolling-horizon planner acting on the D stored *real* days (replayed exactly through the environment). These planner demonstrations act like the logs of a trusted existing controller. Seeding an off-policy replay buffer with demonstrations is established practice for cutting the cost of early exploration (Hester et al., 2018; Vecerik et al., 2017) but what differs here is the source, since the demonstrator is an exact planner rather than a human operator. This is to prevent the battery from untrained random actions. They put real-market transitions and sensible behaviour into the buffer without the policy ever having to act on the real system before it's trained. From then on, all exploration happens in imagination. Checkpoints are chosen using only each arm's own data source, the real simulator is used only for the final held-out evaluation.

E. The Adaptation Loop

The agent is deployed in a streaming setup where it sees one real market day at a time and an adaptation step runs at the end of each day. Each night, the adaptive agent does the following.

1) **Observes and stores** the day's 24 published prices, adding them to its history. This data arrives for free, no matter what the policy did.
2) **Detects** how surprising the day was, by computing the negative log-likelihood (NLL) of the observed day under the ensemble, the detect-then-adapt pattern of the concept-drift literature (Gama et al., 2014) with likelihood under the current model as the drift statistic. A regime shift surprises every ensemble member at once, so this per-day NLL is a clearer signal than measuring how much the members disagree. The NLL over the whole deployment is itself one of the reported results. (The main version adapts every night regardless; a version that only adapts when the NLL is high is a compute-saving refinement.)
3) **Refits the market model.** It fine-tunes each ensemble member, starting from its current weights, on the most recent 90 days of observed prices plus a freshly-resampled set of 30 pre-deployment "anchor" days that guard against forgetting, the plasticity-stability trade-off that continual learning has to manage (Khetarpal et al., 2022). It's deliberately *not* retrained from scratch, because much of the old structure (the daily shape, calendar effects) still holds and should be kept. Each member resamples its data (bootstrap) so the ensemble stays diverse.
4) **Refreshes demonstrations.** It runs the rolling-horizon planner on the last 7 *real* (post-shift) days and adds those transitions to a small replay buffer, so the policy update is grounded in real shifted behaviour and not only in imagination.

5) **Fine-tunes the policy in imagination.** It continues SAC training from the current weights for a fixed number of steps inside the updated learned market. The imagination environment refers to the ensemble directly, so every generated day already reflects the newly refitted model.
6) **Deploys the next day.** All of the steps above run off the physical asset, overnight.

The model-free alternative, which fine-tunes SAC directly on the real price stream, is limited to just 24 new transitions per day. The gap between these two approaches is exactly what the experiment measures.

VI. Experimental Setup

A. Experiment 1: Sample Efficiency under a Real-Data Budget

Each arm is given exactly D days of real market history and nothing more.

- **online.** Standard SAC trained on the real simulator and stopped after $24 \cdot D$ steps (the number of hours in D days). This shows what plain model-free learning can extract from the budget.
- **replay.** SAC trained on the same D stored days replayed exactly, but with unlimited gradient updates. This is the control that separates the effect of "more gradient updates" from the effect of "having a generative model".
- **dyna.** SAC trained in the learned market that was fitted to the same D days (when the gate allows it), with the replay buffer seeded by planner demonstrations on those days (Section V-D).

All arms are evaluated the same way, over 20 paired 14-day episodes on the real simulator, and reported as a percentage of the hindsight optimum. Varying D shows how both policy performance and gate pass/fail depend on the amount of data. An *unlimited-data* SAC reference, trained far beyond any budget, marks the model-free ceiling.

B. Experiment 2: Zero-Shot Robustness to Transient Shocks

The budget-trained policies are frozen and then tested under the five scenario types from Section III-F, applied as short shock windows inside the evaluation episodes. These are regimes the policies never saw during training. Because the scenarios consume no randomness, each shocked episode can be paired with a calm episode from the same seed, which isolates the effect of the shock exactly. The question isn't whether performance drops (it drops for everything) but how gracefully each policy degrades compared with the oracle on the same shocked days.

C. Experiment 3: Adaptation to Persistent Regime Shifts

Each deployment is one continuous 120-day market run. The first 30 days are calm and establish each arm's baseline then a persistent shift begins at day $T = 30$ and lasts until the end. There are three shifts, chosen to span a simple level change versus a structural change.

- **fuel-step.** The gas price doubles ($\times 2$) permanently. This is a pure change in price *level*, which the 24-hour observation window already conveys quite well, so the frozen policies should cope best here.
- **capacity-loss.** The cheap 100 MW baseload coal unit (with its minimum-run block) is permanently lost. This is a *structural* change to the merit order that alters *when* scarcity and negative prices occur, not just how high prices are.
- **cold-regime.** A lasting increase in heating demand, which shifts both the level of demand and its seasonal pattern.

The five arms compared in each deployment are:

- **frozen-dyna** and **frozen-ideal.** The dyna and unlimited-data policies, never updated during the deployment.
- **online-ft.** The unlimited-data policy fine-tuning directly on the real price stream, which provides only 24 new transitions per day.
- **adaptive-dyna.** The full nightly adaptation loop of Section V-E.
- **heuristic.** The fixed price-threshold rule of Section V-A.

All arms, together with the rolling-horizon **oracle**, run on the same paired seeds. Prices are exogenous, so any two arms sharing a seed see exactly the same market.

All metrics are normalized by the oracle on the same seed. Because the oracle replans on each day's observed prices, it adapts instantly by construction, which makes it a fair "instant adaptation" ceiling.

- **capture**(t), the policy's net profit over a trailing 7-day window divided by the oracle's, i.e. the fraction of the instantly-adapted optimum the policy keeps;
- **recovery lag**, how many days after T it takes for capture to return to 90% of the arm's *own* pre-shift level;
- **cumulative regret**, the total post-shift shortfall against the oracle, in dollars;
- **detection trace**, the adaptive arm's per-day NLL, before and after each nightly fine-tune, showing whether and when the model noticed the shift.

D. Hyperparameters, Seeds, and Reporting

All SAC arms use Stable-Baselines3 (Raffin et al., 2021) defaults, except warmup, which scales with the step budget ($\min(2{,}000, \max(T/10, 100))$) so the small online budgets aren't burned entirely on random actions. Replay and dyna train for 500,000 steps across 4 parallel envs, checkpointed by periodic 5-episode evals on each arm's own data source (Section V-D). The market ensemble (Section V-B) is 5 GRUs, hidden width 128, 2,000 epochs. The nightly loop (Section V-E) fine-tunes for 25 epochs and trains the policy for 10,000 imagined steps; online-ft instead takes 240 real gradient steps a night. Seed ranges never overlap: training at seed 0, held-out eval from seed 1,000,000, adaptation deployments from seed 7,000,000.

Dollar amounts scale with the simulator's price cap, because most arbitrage value comes from capturing scarcity hours. We measured this sensitivity rather than assuming it. The hindsight optimum is about +\$90 per 14-day episode at a \$3000/MWh cap and +\$28 at \$1000, and at \$200 the heuristic falls below idle because spread arbitrage alone no longer covers the cost of cycling the battery. We fix the cap at \$1000/MWh for all experiments, chosen so the benchmark ladder stays strictly ordered and evaluation variance stays manageable. Every headline number is reported as a fraction of the oracle or hindsight optimum on the same seeds, which cancels out this level scaling. The learned-policy comparisons are all run at this single cap; Section VIII-B states what is and isn't claimed as a result.

VII. Results and Discussion

A. Does a Validated Model Buy Sample Efficiency?

Unless stated otherwise, quoted percentages are of the hindsight optimum on the real simulator (20 paired 14-day episodes for the budget and robustness experiments), and of the same-seed rolling-horizon oracle (10 paired 120-day deployments) for the adaptation experiment. Because these are ratios of pooled sums rather than means of per-episode ratios, their error margins are leave-one-out jackknife standard deviations over the resampling unit named in each caption. Quantities that are not pooled ratios, namely regret in dollars and surrogate error across fit seeds, carry plain standard deviations instead.

Table I sweeps $D \in \{90, 365, 1095\}$ real days for the hour-autoregressive ensemble, and only $D = 365$ passes every criterion. At $D = 90$ it fails in the expected direction, since barely a season means the invented daily spread comes out more than double the real year's and the winter/evening patterns flip sign entirely. At $D = 1095$ it fails differently. Pooling three winters of varying severity smooths away exactly the seasonal extremity one well-covered year preserves. So the gate isn't rewarding "more data" as such, it's picking out one specific window, and neither more nor less data was assumed safe going in.

TABLE I
Validation-gate outcome across real-data budgets D (days), hour-autoregressive $K = 5$ ensemble. Synthetic value shown per cell; ✓/× marks whether the criterion passes (Section V-C). Every row is judged against an absolute threshold except mean daily spread, which is judged relative to the real column (within 30%).

Metric	Real	D=90	D=365	D=1095
Median (\$/MWh)	67.8	63.1 ✓	62.4 ✓	61.6 ✓
Winter median gap	8.1	11.6 ✓	12.2 ✓	10.3 ✓
Winter mean gap	27.5	−44.3 ×	17.1 ✓	10.4 ×
Evening − night	51.0	−19.7 ×	31.1 ✓	32.5 ✓
Mean daily spread	84.5	185.8 ×	91.1 ✓	72.7 ✓
Cap share	0.005	0.007 ✓	0.002 ✓	0.002 ✓
Negative share	0.007	0.007 ✓	0.002 ✓	0.001 ✓
Gate		**FAIL**	**PASS**	**FAIL**

At $D = 365$, the only budget that passes, dyna hits $71.0 \pm 14.8\%$ of hindsight (best checkpoint). Replay, on identical data and gradient budgets, reaches $38.8 \pm 32.3\%$ at the same budget (and at most 54.1% at any budget), behind dyna by more than either margin, so it's the generative model doing the work, not extra gradient steps. The unlimited-data reference hits $72.5 \pm 13.1\%$; the gap between it and dyna is well inside one jackknife standard deviation, so one year of real data through the learned market recovers the model-free ceiling, statistically, on two orders of magnitude less real history (Figure 1). Dyna is a single point rather than a curve, since the gate refused the other two budgets; online and replay both improve with more data but stay well below the oracle across the whole range.

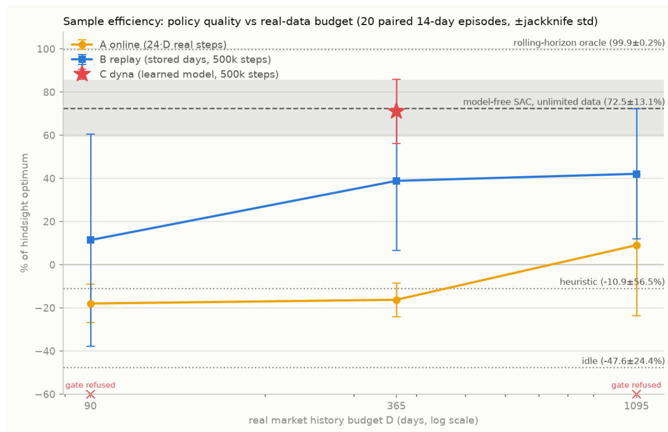

Fig. 1. Policy quality (% of hindsight optimum) versus real-data budget D, 20 paired 14-day episodes per point. Dyna exists only at $D = 365$, the only budget its market model passed the validation gate at.

B. On-Asset Feasibility: Training-Time Exploration versus Deployment

Only the rolling-horizon planner generates real transitions during Dyna training, and it solves a dynamic program over an SoC grid confined to $[\mathrm{soc\_min}, \mathrm{soc\_max}]$, so an infeasible move is never in its search space to begin with. SAC's own exploration still tries infeasible actions constantly, since that's how RL exploration works, but only inside the imagined market against the exact battery physics, never against the real asset. So training doesn't avoid violations, it just keeps them off the physical battery. Table II (top block) confirms this over 20 paired 14-day episodes. A model-free agent exploring on the asset requests something infeasible on $35.8 \pm 3.3\%$ of its steps, mostly during warmup; even the heuristic hits $29.4 \pm 0.1\%$ while the planner hits zero.

But that guarantee doesn't survive deployment. Table II's bottom block shows all three trained checkpoints requesting an SoC-throttled action on over 90% of steps, and dyna-365 ($92.8 \pm 6.7\%$) is statistically indistinguishable from replay-365 ($95.2 \pm 5.8\%$) and the fully model-free unlimited-data policy ($95.1 \pm 4.9\%$).

Why? The natural suspicion is careless training, but a follow-up measurement points elsewhere. We recorded, for the same episodes, the share of steps each deployed policy spends with its SoC already within 0.1 of an operating bound (the table's final column), and it tracks the request rate almost exactly: $95.6 \pm 4.4\%$ for dyna-365, and indistinguishably for the other two arms. A trained policy learns to sit near full or near empty, since that's what buying low and selling high looks like; once

TABLE II

INFEASIBLE ON-ASSET ACTION REQUESTS (AN SOC BOUND THROTTLING MORE THAN 1% OF THE REQUESTED ENERGY), 20 PAIRED 14-DAY EPISODES ON THE REAL MARKET, MEAN ± STD OVER EPISODES. THE MIDDLE COLUMN EXTRAPOLATES TO A 365-DAY BUDGET OF ON-ASSET STEPS AT THE MEAN RATE. THE TOP BLOCK LISTS BEHAVIOURS THAT WOULD SOURCE REAL TRANSITIONS DURING TRAINING, THE BOTTOM BLOCK THE FULLY TRAINED POLICIES AS RUN AT EVALUATION; FOR THOSE, THE FINAL COLUMN GIVES THE SHARE OF STEPS SPENT PARKED WITHIN 0.1 OF AN SOC BOUND, THE FOLLOW-UP MEASUREMENT THAT ATTRIBUTES THE REQUEST RATE.

On-asset behaviour	Infeasible-request rate	Per 365-day budget	Near-bound share
Random (model-free warmup exploration)	$35.8 \pm 3.3\%$	≈3,100	n/a
Heuristic (fixed rule)	$29.4 \pm 0.1\%$	≈2,600	n/a
Rolling-horizon planner (Dyna data source)	$0.0 \pm 0.0\%$	0	n/a
Idle	$0.0 \pm 0.0\%$	0	n/a
Trained dyna-365 (deployed, deterministic)	$92.8 \pm 6.7\%$	≈8,129	$95.6 \pm 4.4\%$
Trained replay-365 (deployed, deterministic)	$95.2 \pm 5.8\%$	≈8,343	$96.2 \pm 4.3\%$
Trained unlimited-data (deployed, deterministic)	$95.1 \pm 4.9\%$	≈8,334	$96.5 \pm 3.4\%$

parked at a bound, almost no room is left, so any further push in the same direction gets throttled. The near-coincidence of the two columns says the requests are overwhelmingly of this parked kind, pushes whose executed effect is a no-op, rather than deep violations mid-range. That is also why the training method stops mattering: the cause is ordinary converged-policy behaviour, not careless training, so there's no reason the safer, off-asset pipeline would avoid it any more than the other two arms do. So RQ2's answer splits cleanly. During data collection the answer is yes, and it's worth roughly 3,100 avoided infeasible requests per training year (Table II, top block). After deployment the answer is no, but the follow-up shows the residual requests are bound-parking pressure rather than dangerous excursions; what a real battery-management system would make of ~8,000 such refused pushes a year, benign or not, is a deployment question this simulator can't settle (Section VIII-B).

C. Does the Model Degrade Gracefully Under Shocks?

Yes it does but not perfectly. Frozen dyna-365 keeps $58.1 \pm 6.4\%$ to $90.0 \pm 2.6\%$ of the oracle's capture depending on the shock (Table III), tracking the unlimited-data policy shock for shock on a real-data budget two orders of magnitude smaller; no shock separates the two arms by more than their jackknife margins (under the fuel shock dyna's point estimate is even ahead, $58.1 \pm 6.4\%$ vs. $56.4 \pm 6.4\%$, though the margin covers it). Fuel-shock is hardest for every arm, since it perturbs the price process in a way the 24-hour window conveys less directly. Cold-snap and plant-outage, by contrast, actually leave dyna-365 and unlimited-data *above* their calm capture, because those shocks mostly raise a price level the window already shows. Replay is the weakest arm throughout, and by margins its wide uncertainty can't excuse (its best shock, $47.1 \pm 14.8\%$, sits below dyna's worst), which reads as a general policy-quality gap rather than a shock-specific one. The calm-trained controller is therefore a floor rather than a solution: it doesn't need shock exposure to avoid disaster, but the fuel-shock column shows how much a level-shifting regime still costs.

D. Can the Adaptation Loop Recover from Persistent Shifts?

No, across all three shifts and 10 paired deployments each, adaptive-dyna never beats simply freezing the pre-shift policy (Table IV). Both arms run identical deployments, so the paired difference is the test: frozen-dyna's post-shift capture exceeds adaptive-dyna's by 16.3 ± 3.5, 11.0 ± 1.4 and 8.9 ± 1.7 percentage points on fuel-step, capacity-loss and cold-regime. Adaptive-dyna is behind before the shift even begins, by 22.0 ± 4.1 points on fuel-step. Table V splits that deficit in two: an acute startup transient over the first fortnight, while the ensemble fine-tunes on a data-starved history, and a smaller steady-state penalty that never closes.

None of the pre-registered hypotheses (Section VI-C) survive. Recovery lag orders differently on every shift, and is measured against an arm's *own* pre-shift level, so adaptive-dyna "recovering" only means returning to an already-worse baseline. Frozen-dyna copes worst with the pure level shift predicted to suit it best (53.4% on fuel-step) and best with the structural shift predicted to be hardest (88.7% on capacity-loss). The NLL trace spikes as often before the shift as after, so it's too noisy to trigger anything on its own.

1) Where the shortfall comes from: The two suspects are the nightly market refit and the policy fine-tune. To separate them we replayed the refit offline against the fuel-step deployment, using the loop's exact configuration (90-day window, 30 anchor days, 25 epochs), and re-scored the ensemble after every night against the same gate that admitted the original 365-day fit.

The refit leaves the gate on its third night and never returns across the remaining 116 (`scripts/measure_ensemble_drift.py` reproduces the full trace). The original ensemble's mean daily spread (83.6) sits inside the required band (59.2–109.9); by night three it's at 141.3. The shift doesn't begin until day 30, so the model has already drifted out of calibration 27 days before the market changes at all. Nightly refitting degrades the ensemble on stationary data, which is also why the adaptive arm trails the frozen one *before* the shift (Table V), when there's nothing to adapt to.

That timing also rules out the obvious repair. Re-gating the nightly refit, freezing the ensemble on any night it fails, would freeze it on night three; and an adaptive arm whose model stops updating on night three is frozen-dyna, the arm that already wins. The gate can prevent this failure but can't convert it into successful adaptation. Incremental nightly fine-tuning on

TABLE III

ZERO-SHOT CAPTURE (% OF THE SAME-REGIME ROLLING-HORIZON ORACLE, POOLED RATIO ± LEAVE-ONE-EPISODE-OUT JACKKNIFE STD) UNDER THE CALM CONTROL AND FIVE TRANSIENT SHOCKS, 20 PAIRED 14-DAY EPISODES, SEED0= 1,000,000 (SCRIPTS/MEASURE_SCENARIO_ROBUSTNESS.PY). NONE OF THE THREE POLICIES SAW ANY SHOCK DURING TRAINING.

Policy	Calm	Cold-snap	Heat-wave	Fuel-shock	Drought	Plant-outage
dyna-365	71.1 ± 14.8	90.0 ± 2.6	83.1 ± 8.2	58.1 ± 6.4	72.9 ± 18.2	87.3 ± 3.5
replay-365	38.9 ± 32.4	43.7 ± 6.9	33.1 ± 18.2	19.1 ± 9.7	43.0 ± 43.4	47.1 ± 14.8
unlimited-data	72.6 ± 13.1	90.8 ± 1.9	83.0 ± 7.5	56.4 ± 6.4	72.1 ± 18.3	89.4 ± 2.3

TABLE IV

ADAPTATION RESULTS, ALL THREE SHIFTS, 10 PAIRED DEPLOYMENTS EACH. CAPTURE IS POOLED 7-DAY-WINDOWED NET PROFIT OVER THE SAME-WINDOW ORACLE NET PROFIT, ± LEAVE-ONE-DEPLOYMENT-OUT JACKKNIFE STD; REGRET IS MEAN ± STD OVER DEPLOYMENTS; RECOVERY LAG IS DAYS AFTER T UNTIL AN ARM REGAINS 90% OF ITS *own* PRE-SHIFT CAPTURE (NEVER = NOT WITHIN THE 90-DAY POST-SHIFT HORIZON), COMPUTED ON THE POOLED CURVE (SCRIPTS/ADAPTATION_STATS.PY).

Shift	Arm	Pre capture (%)	Post capture (%)	Recovery lag	Regret ($)
fuel-step	adaptive-dyna	66.5 ± 10.3	37.1 ± 7.2	19d	231 ± 85
fuel-step	frozen-dyna	88.5 ± 8.0	53.4 ± 8.1	20d	172 ± 47
fuel-step	frozen-ideal	91.8 ± 5.0	41.6 ± 8.7	never	214 ± 74
fuel-step	online-ft	85.0 ± 5.8	12.3 ± 8.3	never	328 ± 193
fuel-step	heuristic	54.7 ± 25.3	27.1 ± 9.5	14d	282 ± 111
capacity-loss	adaptive-dyna	63.4 ± 14.3	77.7 ± 4.6	1d	218 ± 119
capacity-loss	frozen-dyna	90.2 ± 7.2	88.7 ± 3.5	7d	110 ± 54
capacity-loss	frozen-ideal	90.5 ± 5.1	91.1 ± 2.1	1d	90 ± 55
capacity-loss	online-ft	78.2 ± 6.7	66.0 ± 6.6	9d	348 ± 301
capacity-loss	heuristic	54.5 ± 25.1	55.2 ± 6.2	3d	467 ± 309
cold-regime	adaptive-dyna	71.2 ± 17.4	74.0 ± 3.1	8d	857 ± 858
cold-regime	frozen-dyna	90.3 ± 7.2	82.9 ± 4.2	1d	546 ± 695
cold-regime	frozen-ideal	90.6 ± 5.1	87.6 ± 2.0	1d	394 ± 443
cold-regime	online-ft	80.4 ± 7.7	83.8 ± 1.6	1d	538 ± 412
cold-regime	heuristic	54.5 ± 25.1	69.8 ± 1.7	1d	982 ± 851

TABLE V

ADAPTIVE-DYNA'S SETTLED-CALM CAPTURE (7-DAY WINDOWS FULLY INSIDE DAYS 20–29, AFTER THE STARTUP TRANSIENT) VERSUS FROZEN-DYNA, AND BOTH ARMS' SETTLED POST-SHIFT CAPTURE (WINDOWS FULLY INSIDE DAYS 37–119, AFTER THE SHIFT'S OWN SETTLING WINDOW). POOLED CAPTURE ± JACKKNIFE STD, IN %.

Shift	Adapt. (calm)	Frozen (calm)	Adapt. (post)	Frozen (post)
fuel-step	83.7 ± 9.0	94.0 ± 5.1	40.7 ± 7.9	55.6 ± 8.8
capacity-loss	80.3 ± 11.0	93.4 ± 3.8	78.5 ± 3.4	89.7 ± 2.4
cold-regime	81.8 ± 9.9	93.8 ± 3.7	75.1 ± 3.1	82.3 ± 4.5

a rolling window inflates the model's price spreads whether or not the market has moved, and SAC then trains inside a model whose swings are systematically larger than the real market's, so the policy is shaped to exploit spreads that don't exist. The earliest refits have the least data and are the most miscalibrated, which is the startup transient.

E. Does a Physics Prior Need Fewer Transitions?

No, and not even in the small-N regime a PINN is meant to own (Section II-D, Table VI). Holding the market exact, an MLP and a physics-informed network both learn $(\mathrm{soc}, \mathrm{soh}, \text{power fraction}) \mapsto (\Delta\mathrm{soc}, \Delta\mathrm{soh}, \text{grid energy})$ from a shrinking budget of N transitions, over ten fit seeds. The soft residual beats the MLP only at N=500 (0.014 ± 0.002 vs. 0.018 ± 0.003), and its apparent edge at N=200 sits inside one standard deviation. Below that the MLP wins outright and by widening margins, the battery map being smooth enough that an unconstrained network interpolates it from a handful of points. Under a narrow training distribution the MLP wins at every budget.

TABLE VI

HELD-OUT POOLED RELATIVE MAE (ABSOLUTE ERROR OVER TARGET STD, POOLED ACROSS THE THREE TARGETS), UNIFORM COVERAGE, MEAN ± STD OVER TEN FIT SEEDS. **BOLD** IS BEST POINT ESTIMATE AT EACH BUDGET. PINN-HARD IS THE SAME TRUNK AS MLP WITH THE OUTPUTS CLAMPED TO PHYSICALLY POSSIBLE VALUES AND NO RESIDUAL, SO THE MLP TO PINN-HARD STEP ISOLATES THE CLAMP AND THE PINN-HARD TO PINN-SOFT STEP ISOLATES THE PHYSICS RESIDUAL.

N	mlp	pinn-hard	pinn-soft
500	0.018 ± 0.003	0.020 ± 0.005	$\mathbf{0.014 \pm 0.002}$
200	0.034 ± 0.003	0.041 ± 0.005	$\mathbf{0.032 \pm 0.003}$
100	$\mathbf{0.043 \pm 0.003}$	0.058 ± 0.006	0.065 ± 0.006
50	$\mathbf{0.076 \pm 0.006}$	0.128 ± 0.010	0.170 ± 0.020
25	$\mathbf{0.147 \pm 0.009}$	0.151 ± 0.014	0.187 ± 0.016
15	$\mathbf{0.358 \pm 0.014}$	0.498 ± 0.043	0.521 ± 0.011
10	$\mathbf{0.454 \pm 0.029}$	0.648 ± 0.033	0.504 ± 0.013

The unconstrained MLP does predict impossible transitions, up to $17.9 \pm 3.3\%$ of test points at N=10, but the output clamp that rules these out isn't free either: `pinn-hard` is the `mlp` arm with clamped outputs, and it's the less accurate of the two at every budget.

1) Wrong-constraints ablation: To be thorough about *why* the prior doesn't pay, we asked whether a PINN given the

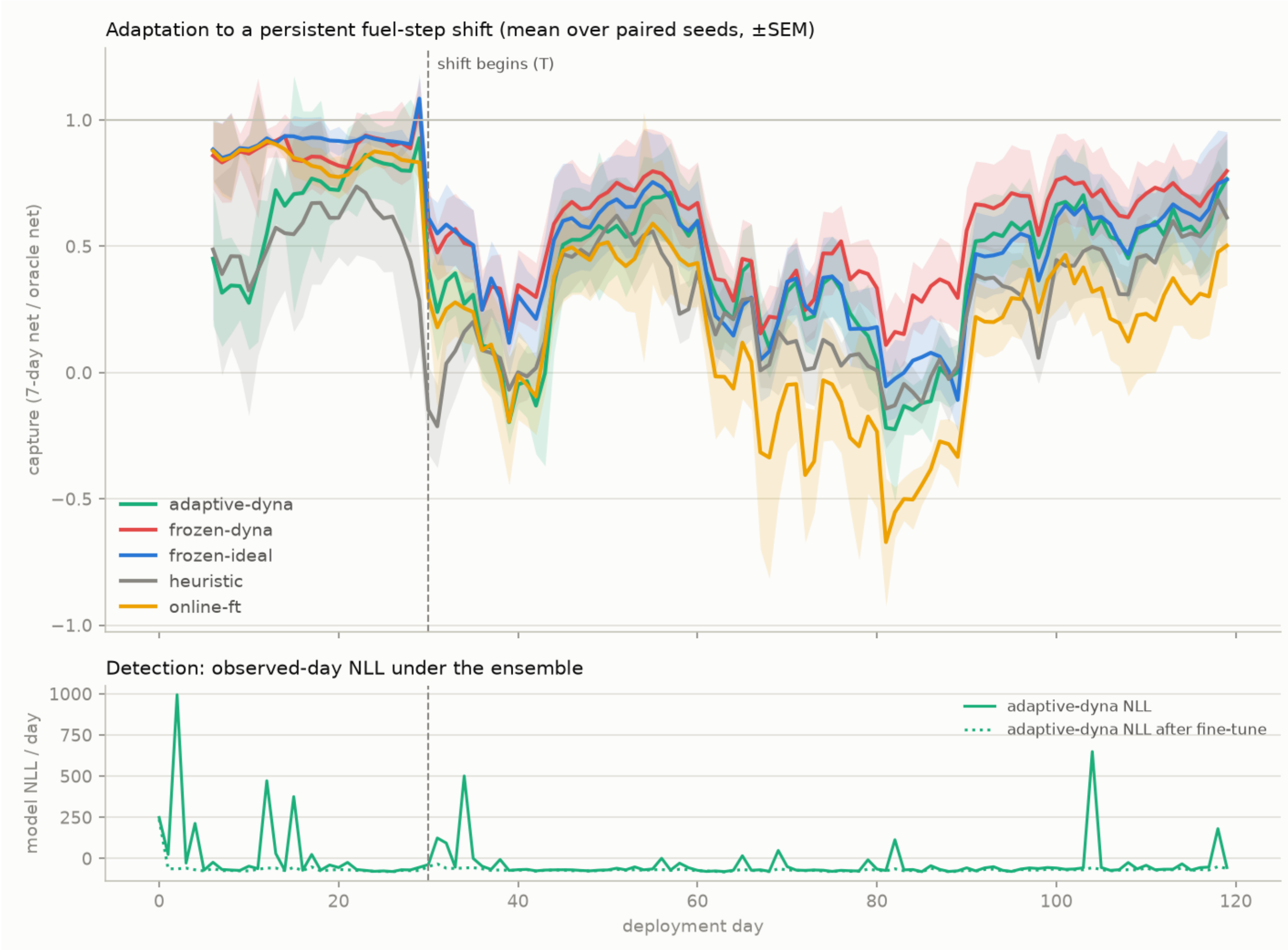

Fig. 2. Capture curves and NLL detection trace, fuel-step shift, mean over 10 paired seeds (±SEM).

wrong physics could still beat the MLP. If it could, the residual was never carrying physical information and its occasional win was generic regularization. We refit the same tuned soft-residual PINN three times with a broken law: efficiency frozen at 0.60 instead of the true 0.949, nameplate capacity halved in the energy identity, and the calendar-ageing law flipped so an empty battery ages fastest. A control freezes efficiency at its *true* value, separating "frozen" from "wrong".

It can't (Table VII). At $N{=}500$ wrong efficiency (0.070 ± 0.004) and wrong capacity (0.071 ± 0.002) cost about $4.9\times$ the correct residual's error and land about $3.9\times$ worse than the MLP, while the control (0.014 ± 0.002) matches the correct residual exactly. So where the residual helps at all, it helps because the physics is right. But the penalty for wrong physics shrinks steadily as data thins ($4.9\times$, $2.7\times$, $1.6\times$, $1.1\times$ at $N = 500, 200, 100, 50$) and disappears below $N{=}50$, where every wrong variant sits inside the seed spread of the correct one.

The residual is therefore inert exactly where a PINN is supposed to help. It only bites once there's enough data to identify its physical parameters, which is the regime where the plain MLP was already good enough. Taken with the accuracy results, the verdict on this problem is simply negative: the physics prior isn't practical here. It loses on accuracy at every budget an operator would realistically have, its residual carries no information at those budgets, and its one apparent guarantee is an output clamp restated, available to any network without physics. Section VIII-B notes what would have to change for the conclusion to differ.

VIII. Conclusion and Future Work

A. Summary of Findings

The results show that watching the market for a single year is enough. Feed that year into a generative model of daily prices, check the model against held-out data, and train the agent inside it, and the resulting policy is as good as one trained on unlimited real market experience ($71.0 \pm 14.8\%$ of the hindsight optimum against $72.5 \pm 13.1\%$). The comparison that matters is the replay control, which sees exactly the same year and gets as many gradient updates as it wants, and still

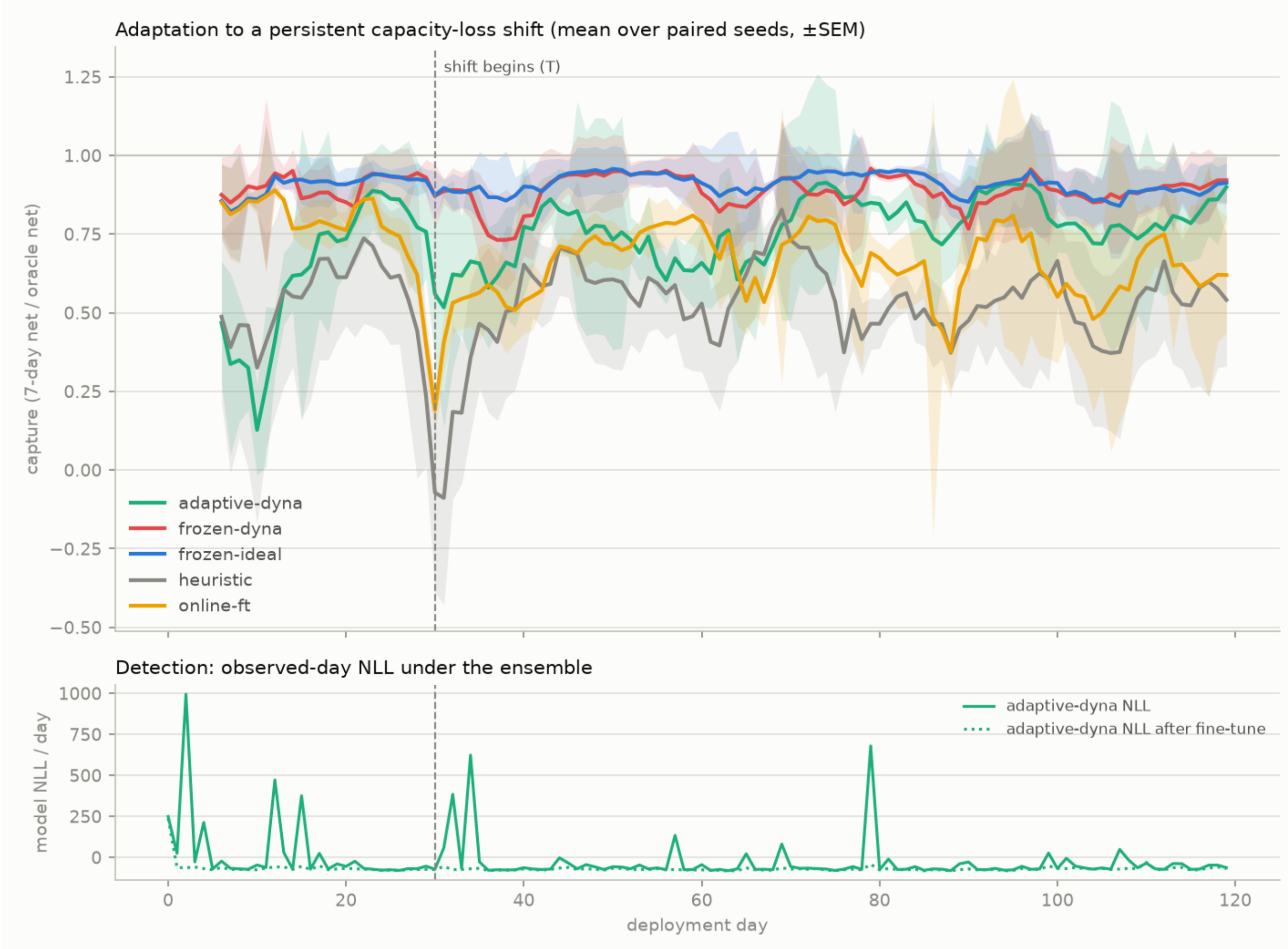

Fig. 3. Capture curves and NLL detection trace, capacity-loss shift, mean over 10 paired seeds (±SEM).

lands 32 points behind. So the gain doesn't come from training harder on the same data. It comes from having a model good enough to invent new days to train on. And "good enough" isn't a judgement call here: the gate decides it against fixed criteria, before any training compute is spent.

Beyond that, the dissertation maps where the same decomposition stops paying. Over hours there's nothing to fix, since the 24-hour price window already carries almost all the value and the rolling-horizon oracle reaches 99.9% of hindsight. Over days, a policy trained only on calm data holds up when a shock it has never seen arrives, keeping 58 to 90% of what the oracle makes on the same shocked days. That is a floor rather than a solution, but it comes without any shock training.

Adaptation is where it breaks, and not for the reason we expected (Section VII-D). The difficulty isn't noticing that the market has moved. It's that refitting the model every night on a rolling window quietly degrades it, even while the market is still stationary. The model leaves its own calibration band on the third night, 27 days before the shift arrives. Keeping a model trustworthy while it goes on learning turns out to be a harder requirement than getting it right once. A stricter gate doesn't solve this because a gate can only reject a bad refit, and rejecting every refit is the same thing as freezing the model. What the loop needs is a refit that holds on to what the original year taught it while it absorbs new days, instead of drifting away from both (Section VII-D1).

The battery-side study results in a similar direction where learning the battery instead of assuming it gives no sample-efficiency gain over a plain network, and the wrong-constraints ablation shows why. The physics residual only carries information when data is plentiful, and goes inert below $N{=}50$, exactly where it was meant to help (Section VII-E1). Its zero rate of impossible predictions isn't a result but a restatement of its output clamp, which any network can use. The honest verdict is that the physics prior isn't practical on this problem. And model-free online learning is the weakest option throughout, taking years to converge from scratch and coming out worst after a shift. The through-line is that it's the world model's trustworthiness, not raw interaction, that carries this approach.

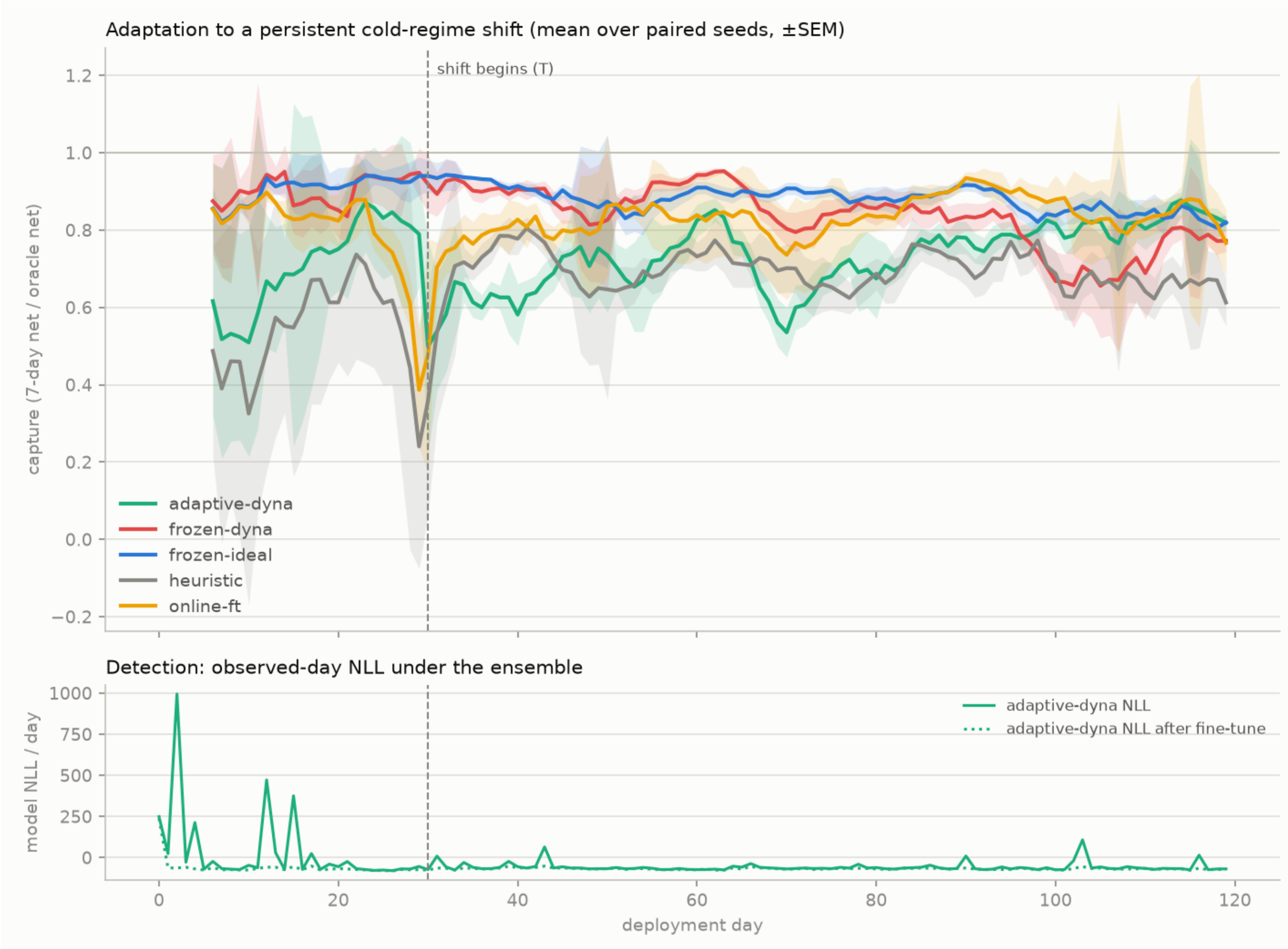

Fig. 4. Capture curves and NLL detection trace, cold-regime shift, mean over 10 paired seeds (±SEM).

B. Limitations

Everything here is simulation, a synthetic market and battery. It's a controlled testbed, and none of this transfers to a real deployment without first validating against measured price and cell data. The price-taker assumption does double duty (it justifies both the decomposed world model and the hindsight oracle) and would weaken for storage big enough to move prices. The persistent-shift evaluation covers three hand-picked regimes, not a continuous range or gradual compound drift. The rolling-horizon normalizer adapts instantly only because prices are fully known a day ahead; in a market that relied on forecasts, the ceiling itself would lag. The budget-experiment policies come from a single training seed (though evaluation is multi-seed and paired). And the zero-infeasible-request guarantee of Section VII-B only covers planner-sourced data collection during training; the trained policy itself requests infeasible actions on over 90% of deployment steps. The follow-up measurement attributes these to benign SoC-bound parking (the policy sits at a bound on ∼96% of steps, so the refused pushes are no-ops), but whether a real battery-management system tolerates thousands of refused pushes a year is a deployment question the simulator can't settle. So on-asset safety should be read as solved for training, and characterised but not certified for deployment.

Every quantitative result here also depends on a single setting of the world, one European heating-dominated market calibration, one price cap ($1000/MWh), one battery cost model ($250/kWh replacement, 5,000-cycle life). We measured the cap dependence rather than assumed it away (Section VI-D). Raw dollars scale with the cap since scarcity capture dominates arbitrage value, and the benchmark ladder stays in order from $200 to $3000, except the heuristic dropping below idle at the lowest cap. We didn't repeat the learned-policy comparisons at other caps, calibrations, or degradation prices, so the *sizes* of the reported gaps are specific to this setting. What actually holds up is the orderings and the mechanisms behind them. A validated generative model turns a fixed budget of observed days into unlimited training experience, and free nightly observations turn into off-asset adaptation. Grid constraints such as transmission congestion, and temperature-dependent ageing beyond the SoC-

TABLE VII

WRONG-CONSTRAINTS ABLATION. HELD-OUT POOLED RELATIVE MAE, UNIFORM COVERAGE, MEAN ± STD OVER TEN FIT SEEDS, IDENTICAL TRAINING DRAWS AND TEST SET TO TABLE VI. SOFT-CORRECT IS THE TUNED SOFT-RESIDUAL PINN; SOFT-ETA-TRUE FREEZES EFFICIENCY AT ITS TRUE VALUE (A CONTROL FOR FREEZING ITSELF); THE REMAINING THREE ARMS FREEZE OR FLIP A PHYSICAL LAW TO A WRONG VALUE. **BOLD** MARKS THE BEST POINT ESTIMATE IN EACH ROW. THE CORRECT-VERSUS-WRONG GAP IS LARGE AT HIGH BUDGETS AND VANISHES BELOW N=50, WHERE THE RESIDUAL STOPS INFLUENCING THE FIT AT ALL.

N	mlp	soft-correct	soft-eta-true	soft-eta-wrong	soft-cap-wrong	soft-cal-wrong
500	0.018 ± 0.003	0.014 ± 0.002	$\mathbf{0.014 \pm 0.002}$	0.070 ± 0.004	0.071 ± 0.002	0.022 ± 0.002
200	0.034 ± 0.003	0.032 ± 0.003	$\mathbf{0.031 \pm 0.003}$	0.086 ± 0.005	0.082 ± 0.003	0.036 ± 0.003
100	$\mathbf{0.043 \pm 0.003}$	0.065 ± 0.006	0.065 ± 0.007	0.104 ± 0.005	0.100 ± 0.009	0.067 ± 0.008
50	$\mathbf{0.076 \pm 0.006}$	0.170 ± 0.020	0.166 ± 0.018	0.188 ± 0.018	0.158 ± 0.010	0.171 ± 0.019
25	$\mathbf{0.147 \pm 0.009}$	0.187 ± 0.016	0.194 ± 0.018	0.206 ± 0.012	0.217 ± 0.021	0.188 ± 0.019
15	$\mathbf{0.358 \pm 0.014}$	0.521 ± 0.011	0.523 ± 0.008	0.538 ± 0.011	0.539 ± 0.021	0.522 ± 0.012
10	$\mathbf{0.454 \pm 0.029}$	0.504 ± 0.013	0.505 ± 0.016	0.527 ± 0.018	0.494 ± 0.009	0.504 ± 0.012

dependent calendar term, aren't modelled.

C. Future Directions

The clearest next step is to repair the nightly refit, and Section VII-D1 says where to aim. Because the ensemble drifts on stationary data, the thing to fix is the refit itself, not the test wrapped around it. Refitting from scratch on the pooled history each night, instead of fine-tuning from the previous night's weights, would stop the spread inflation compounding. Anchoring more heavily on pre-deployment days would do the same, as would scoring the refit against the calibration statistics rather than likelihood alone. Re-running the three-shift grid with a repaired refit is the natural follow-up.

Two smaller variants are worth running beside it. A re-gated loop, freezing the ensemble on any night it fails the gate, is useful as a control rather than a fix: on this evidence it would freeze within days and collapse into the frozen arm, but it would separate how much of the deficit is model drift from how much is the policy fine-tune. Delaying adaptation until enough days have accumulated is the other, since the worst drift is in the first few nights.

When to adapt is a separate question from how. An NLL-triggered loop could replace the always-adapt default, though per-day NLL is noisy enough before the shift that it would likely make a weaker trigger than the calibration gate; what it saves is compute, and when that's safe is still open. A softer version scales each night's effort with how surprising the day was. The adaptive arm also needs a transient-shock control, since adaptation should do no harm during a one-day crisis, and the nightly update could mix real-day replay with imagination weighted by uncertainty.

On the market model itself, a mixture-density day model might pass the gate at smaller budgets than the current ensemble manages, and a gate whose acceptance regions come from the market's own year-to-year variability would be better calibrated than fixed bands.

The battery study (Section VII-E) closes off more than it opens. Because the residual is inert at realistic log sizes and the output clamp needs no physics, the question worth keeping isn't about physics priors at all. It's whether constraining a surrogate's outputs to be physically possible protects a downstream policy from model exploitation, which can be tested by training policies against a constrained and an unconstrained MLP and evaluating both on the true battery. No physics-informed variant is needed for that. A stronger claim about physics priors would need a harder dynamics problem, since a smooth three-dimensional map is close to the worst case for showing their value. The successes reported on real cells (Wang et al., 2024) are recovered from noisy, partially observed measurements rather than from an exactly simulated step map, so the negative here bounds the claim rather than contradicting those results.

Finally, the evaluation can be widened: validating the simulator's shift scenarios against real market episodes such as the 2022 European price crisis, repeating the budget sweep over several training seeds, modelling battery chemistries whose cycle and calendar ageing balance differently, and relaxing the price-taker assumption toward price-making storage, which would couple the two halves of the world model this dissertation deliberately kept apart.

REFERENCES

Aneke, M. and Wang, M. (2016) 'Energy storage technologies and real life applications: a state of the art review', *Applied Energy*, 179, pp. 350–377. doi: 10.1016/j.apenergy.2016.06.097.

Antonelo, E.A. et al. (2024) 'Physics-informed neural nets for control of dynamical systems', *Neurocomputing*, 579. Available at: https://arxiv.org/abs/2104.02556 (Accessed: 28 July 2026).

Cao, J. et al. (2020) 'Deep reinforcement learning-based energy storage arbitrage with accurate lithium-ion battery degradation model', *IEEE Transactions on Smart Grid*, 11(5), pp. 4513–4521. doi: 10.1109/TSG.2020.2986333.

Cho, K. et al. (2014) 'Learning phrase representations using RNN encoder-decoder for statistical machine translation', in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734. doi: 10.3115/v1/D14-1179.

Chua, K. et al. (2018) 'Deep reinforcement learning in a handful of trials using probabilistic dynamics models', in *Advances* *in Neural Information Processing Systems*, 31. Available at: https://arxiv.org/abs/1805.12114 (Accessed: 28 July 2026).

Gama, J. et al. (2014) 'A survey on concept drift adaptation', *ACM Computing Surveys*, 46(4), pp. 1–37. doi: 10.1145/2523813.

Haarnoja, T., Zhou, A., Abbeel, P. and Levine, S. (2018) 'Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor', in *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)*. Available at: https://arxiv.org/abs/1801.01290 (Accessed: 28 July 2026).

Hafner, D. et al. (2020) 'Dream to control: learning behaviors by latent imagination', in *International Conference on Learning Representations (ICLR 2020)*. Available at: https://arxiv.org/abs/1912.01603 (Accessed: 28 July 2026).

Hester, T. et al. (2018) 'Deep Q-learning from demonstrations', in *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI-18)*, pp. 3223–3230. Available at: https://arxiv.org/abs/1704.03732 (Accessed: 4 August 2026).

International Energy Agency (2022) *World energy outlook 2022*. Paris: IEA. Available at: https://www.iea.org/reports/world-energy-outlook-2022 (Accessed: 4 August 2026).

International Energy Agency (2023) *Energy technology perspectives 2023*. Paris: IEA. Available at: https://www.iea.org/reports/energy-technology-perspectives-2023 (Accessed: 28 July 2026).

Janner, M. et al. (2019) 'When to trust your model: model-based policy optimization', in *Advances in Neural Information Processing Systems*, 32. Available at: https://arxiv.org/abs/1906.08253 (Accessed: 28 July 2026).

Khetarpal, K. et al. (2022) 'Towards continual reinforcement learning: a review and perspectives', *Journal of Artificial Intelligence Research*, 75, pp. 1401–1476. Available at: https://arxiv.org/abs/2012.13490 (Accessed: 28 July 2026).

Lakshminarayanan, B., Pritzel, A. and Blundell, C. (2017) 'Simple and scalable predictive uncertainty estimation using deep ensembles', in *Advances in Neural Information Processing Systems*, 30. Available at: https://arxiv.org/abs/1612.01474 (Accessed: 4 August 2026).

Lecarpentier, E. and Rachelson, E. (2019) 'Non-stationary Markov decision processes, a worst-case approach using model-based reinforcement learning', in *Advances in Neural Information Processing Systems*, 32. Available at: https://arxiv.org/abs/1904.10090 (Accessed: 28 July 2026).

Liu, X.-Y. and Wang, J.-X. (2021) 'Physics-informed Dyna-style model-based deep reinforcement learning for dynamic control', *Proceedings of the Royal Society A*, 477(2255). doi: 10.1098/rspa.2021.0618.

Osband, I., Aslanides, J. and Cassirer, A. (2018) 'Randomized prior functions for deep reinforcement learning', in *Advances in Neural Information Processing Systems*, 31. Available at: https://arxiv.org/abs/1806.03335 (Accessed: 4 August 2026).

Padakandla, S. (2021) 'A survey of reinforcement learning algorithms for dynamically varying environments', *ACM Computing Surveys*, 54(6), pp. 1–25. Available at: https://arxiv.org/abs/2005.10619 (Accessed: 28 July 2026).

Raissi, M., Perdikaris, P. and Karniadakis, G.E. (2019) 'Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations', *Journal of Computational Physics*, 378, pp. 686–707. doi: 10.1016/j.jcp.2018.10.045.

Raffin, A. et al. (2021) 'Stable-Baselines3: reliable reinforcement learning implementations', *Journal of Machine Learning Research*, 22(268), pp. 1–8. Available at: https://jmlr.org/papers/v22/20-1364.html (Accessed: 4 August 2026).

Sioshansi, R., Denholm, P., Jenkin, T. and Weiss, J. (2009) 'Estimating the value of electricity storage in PJM: arbitrage and some welfare effects', *Energy Economics*, 31(2), pp. 269–277. doi: 10.1016/j.eneco.2008.10.005.

Sutton, R.S. (1991) 'Dyna, an integrated architecture for learning, planning, and reacting', *ACM SIGART Bulletin*, 2(4), pp. 160–163. doi: 10.1145/122344.122377.

Vecerik, M. et al. (2017) 'Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards', *arXiv preprint* arXiv:1707.08817. Available at: https://arxiv.org/abs/1707.08817 (Accessed: 4 August 2026).

Wang, F. et al. (2024) 'Physics-informed neural network for lithium-ion battery degradation stable modeling and prognosis', *Nature Communications*, 15(1), 4332. doi: 10.1038/s41467-024-48779-z.

Xu, B. et al. (2018) 'Modeling of lithium-ion battery degradation for cell life assessment', *IEEE Transactions on Smart Grid*, 9(2), pp. 1131–1140. doi: 10.1109/TSG.2016.2578950.